Évaluation — Programmation Avancée & Design Patterns

Mini-projet QueenBee — Master 1 Développement — Durée : 2 heures

PhD Eddy KIOMBA KAMBILO

Nom, prénom : _____ Groupe : _____

Dépôt GitHub : _____

Contexte

Vous rejoignez l'équipe de développement de **QueenBee**, la plateforme de gestion apicole de l'association BeeAfrica (cahier des charges commun M1). Le pilote tourne dans trois pays (Burkina Faso, Cameroun, Sénégal). Le module « Surveillance du rucher » a été écrit dans l'urgence : il fonctionne en partie, mais il est difficile à faire évoluer.

Votre mission : **compléter et assainir ce module** en appliquant les quatre notions vues en séance.

Partie	Notion	Cas du cahier des charges	Points	Temps conseillé
A	Lambda et interfaces fonctionnelles	Seuils d'alerte (administrateur), UC-CY02	3	15 min
B	Streams	Tableau de bord du coordinateur (UC-G01), UC-CY02	6	35 min
C	SOLID (S, O, L, D)	UC-A02 Réception d'une alerte	6	30 min
D	Clean Code et Refactoring	UC-C01 Priorisation des visites	5	25 min
		Total	**20**	105 min + 15 min de lecture et de dépôt

Les quatre parties sont **indépendantes** : les tests d'une partie n'utilisent pas votre code des autres parties. Si vous bloquez, passez à la suite.

Règles

- **Documents autorisés** : supports de cours et de TD du module. Outils d'IA générative interdits.
- **Ne modifiez pas** les fichiers marqués `NE PAS MODIFIER` (données, code « avant », tests). Ils sont comparés automatiquement à l'original : un fichier protégé modifié annule la partie concernée.
- Le projet doit **toujours compiler**. Remplacez chaque ligne

  ```
  throw new UnsupportedOperationException("TODO ...");
  ```

  par votre code ; laissez-la telle quelle si vous ne traitez pas la question.
- **Un commit par partie** au minimum (`git commit -m "Partie A : lambdas"`), et `git push` avant la fin. Seul ce qui est poussé sur GitHub à l'heure de fin est noté.

Démarrage (5 minutes)

1. Clonez votre dépôt et ouvrez-le dans IntelliJ (projet Maven, JDK 21).
2. Lancez `mvn test` (ou clic droit sur `src/test/java` → *Run All Tests*). **Les 28 tests sont rouges : c'est normal.** Votre objectif est de les faire passer au vert.
3. Chaque nom de test commence par le numéro de sa question (`a1_`, `b4_`, `c2_…`). Une question rapporte ses points quand **tous** ses tests passent.
4. `queenbee.eval.Lanceur` affiche une démonstration des quatre parties.

Le jeu de données (`donnees.JeuDeDonnees`) contient 6 ruches sur 3 sites et 12 mesures valides, plus 2 trames anormales : un **rejeu** (trame reçue deux fois) et une trame **hors plage** (85 °C, capteur défaillant).

Partie A — Expressions Lambda (3 points)

Fichier : `partieA/ReglesAlerte.java`

Problème. L'administrateur doit pouvoir configurer des seuils d'alerte. Dans le code d'origine, chaque règle était une classe (`RegleSurchauffeClassique`, `RegleHumiditeClassique`…) : une classe entière pour une seule comparaison. L'interface `RegleAlerte` est maintenant une **interface fonctionnelle** (`@FunctionalInterface`) : chaque règle doit s'écrire en une ligne.

Q.	Travail demandé	Points
A1	`surchauffe()` : Lambda vraie si la température est **strictement** supérieure à 38 °C	0,5
A2	`humiditeExcessive()` : Lambda vraie si l'humidité est **strictement** supérieure à 90 %	0,5
A3	`estPlausible()` : écrire **trois** `Predicate<Mesure>` nommés (température dans $[-10\,;\,60]$, humidité dans $[0\,;\,100]$, poids dans $[0\,;\,150]$, bornes incluses) et retourner leur composition avec `and()`	1,5
A4	`plusRecentesDAbord()` : un `Comparator<Mesure>` qui place les mesures les plus récentes en premier (`Comparator.comparing` + référence de méthode + `reversed()`)	0,5

Utilisez les constantes déjà déclarées : aucun nombre magique.

Partie B — Streams (6 points)

Fichier : `partieB/TableauDeBordRucher.java`

Problème. Le tableau de bord du coordinateur était calculé avec des boucles `for`, des listes temporaires remplies à la main et des `Collections.sort` : long, répétitif, source d'erreurs.

Règle : chaque méthode s'écrit avec **un seul pipeline Stream**, sans `for` ni `while`. Une méthode écrite avec une boucle qui passe le test obtient la moitié des points.

Q.	Méthode	Résultat attendu (opérateurs)
B1	`mesuresValides(Predicate<Mesure> p)`	Mesures qui satisfont le prédicat, rejeux supprimés (`filter`, `distinct`)
B2	`ruchesDuSite(String site)`	Identifiants des ruches du site, triés (`filter`, `map`, `sorted`)
B3	`nombreDeMesuresDeclenchant(RegleAlerte r)`	Nombre de mesures qui déclenchent la règle (`filter`, `count`)
B4	`temperatureMoyenneParRuche()`	`Map` idRuche → température moyenne (`groupingBy`, `averagingDouble`)
B5	`dernierPoids(String rucheId)`	Poids de la mesure la plus récente ; `Optional` vide si aucune mesure (`filter`, `max`, `map`)
B6	`apiculteursDuSite(String site)`	Ex. « Awa Ouedraogo, Issa Kabore » : sans doublon, trié, séparé par une virgule (`map`, `distinct`, `sorted`, `joining`)

Chaque question vaut 1 point. Remarquez que B1 et B3 **reçoivent un comportement en paramètre** : c'est la réutilisation des Lambda de la partie A.

Partie C — SOLID (6 points)

Fichiers : paquet `partieC`

Problème. Lisez d'abord `partieC/avant/ServiceAlertesAvant.java` (UC-A02). Cette classe :

- détecte les risques, envoie les SMS **et** tient le journal → trois raisons de changer (**S**) ;
- impose un nouveau `if` pour chaque nouveau type de risque → elle n'est pas fermée à la modification (**O**) ;
- crée elle-même sa passerelle SMS → impossible de la tester ou de passer au push (**D**).

Lisez aussi `partieC/avant/NotificationPushAvant.java` : elle lève une exception quand le téléphone est hors réseau, alors que le cahier des charges demande « push si connecté, sinon au prochain accès » (**L**).

La nouvelle architecture est déjà dessinée : `RegleRisque`, `EvaluateurRisques` (à ne pas modifier), `CanalNotification` (lisez son **contrat** dans la Javadoc), `CanalSms`, `JournalAlertes`, `Notification`. `RegleSurchauffe` est fournie comme modèle.

Q.	Principe	Travail demandé	Points
C1	O	Écrire `RegleHumiditeExcessive` (> 90 %, nom `"Humidite excessive"`) et le **nouveau besoin** `RegleFroidExtreme` (< 5 °C, nom `"Froid extreme"`), **sans modifier** `EvaluateurRisques`	1,5
C2	S + D	Écrire `ServiceAlertes` : ses trois dépendances sont **reçues par le constructeur** ; `traiter()` envoie et journalise un message `"[idRuche] NomDuRisque"` par risque détecté et retourne leur nombre. Aucun `new` de dépendance dans cette classe	2,5
C3	L	Écrire `CanalPush` qui respecte le contrat : hors réseau, **aucune exception**, la notification est mise en attente ; `seConnecter()` délivre les notifications en attente dans l'ordre	2

Le test de C2 passe au service un **faux canal** qui enregistre les messages au lieu de les envoyer : c'est ce que l'inversion des dépendances rend possible.

Partie D — Clean Code et Refactoring (5 points)

Fichiers : `partieD/avant/PV.java` (code hérité, à ne pas modifier) et `partieD/PriorisationVisites.java`

Problème. `PV.calc` calcule les visites prioritaires d'un site (UC-C01). Le résultat est juste, mais le code accumule les défauts vus en cours : noms d'une lettre, nombres magiques, `String[]` lu par indice, `if` en cascade, tri à la main par double boucle, commentaires inutiles, méthode morte `ancienScore`.

Règle métier : +3 si température > 38 ; +2 si humidité > 90 ; +4 si perte de poids > 2,5 kg ; +1 si dernière visite > 15 jours (seuils **stricts**). Sortie : `"id : score"` pour les ruches du site ayant un score > 0, triées par score décroissant puis par identifiant.

Travail demandé : écrire `PriorisationVisites` (méthodes `visitesPrioritaires` et `scoreDeRisque`) qui produit **exactement la même sortie**, en suivant les étapes du TD4 :

1. **Rename** — des noms qui disent ce qu'ils contiennent.
2. **Extract Constant** — seuils et points deviennent des constantes nommées.
3. **Introduce Parameter Object** — *déjà fait* : utilisez le record `EtatRuche`.
4. **Extract Method** — le calcul du score dans `scoreDeRisque`, des méthodes courtes.
5. **Replace Loop with Pipeline** — filtrage, tri (`Comparator` avec `thenComparing`) et mise en forme en Stream.
6. **Remove Dead Code** — rien de `ancienScore`, aucun commentaire qui répète le code.

Le **test de caractérisation** compare votre sortie à celle de `PV.calc` sur 11 ruches et 4 sites. Il est interdit d'appeler `PV` depuis votre classe.

Critère	Points
D1 — Tests de caractérisation au vert (même sortie que l'ancien code)	2
D2 — Qualité : noms, constantes, méthodes courtes, pipeline Stream, aucun code mort (grille de l'enseignant)	3

Barème récapitulatif

A1	A2	A3	A4	B1–B6	C1	C2	C3	D1	D2	**Total**
0,5	0,5	1,5	0,5	6×1	1,5	2,5	2	2	3	**20**

Avant de partir : `mvn test`, puis `git add .`, `git commit`, `git push`. Vérifiez sur GitHub que votre dernier commit est bien visible.